

♥ MedCore System

Sistema Integrado de Gestão Hospitalar

Trabalho Avaliativo — Tecnologias para Internet (Full Stack)

Modalidade	Full Stack — API + Frontend
Tecnologia	Backend livre · React (ou outro framework SPA)
Entrega	Repositório público (GitHub/GitLab) + deploy opcional
Contexto	Área da Saúde — inspirado em sistemas HIS como o Tasy

1. Contexto e Motivação

Sistemas de Informação em Saúde (HIS — *Health Information Systems*) são a espinha dorsal da gestão hospitalar moderna. Plataformas como o **Tasy** (Philips), o **MV Soul** e o **AGHUse** centralizam informações clínicas, administrativas e financeiras em um único ambiente integrado. Este trabalho propõe o desenvolvimento de um **sistema full stack** — chamado **MedCore System** — que reproduz o núcleo (CRUD) de um HIS real, integrando uma API RESTful com uma interface web responsiva.

O objetivo é fixar conceitos de arquitetura cliente-servidor, consumo de APIs, persistência de dados e boas práticas de desenvolvimento, sempre no contexto prático da área da saúde.

2. Objetivo Geral

Desenvolver uma aplicação **full stack** para gerenciar um sistema de atendimento hospitalar, composta por:

- **Backend** — API RESTful documentada, com persistência em banco de dados relacional ou NoSQL.
- **Frontend** — Interface web (SPA ou MPA) que consome a API e oferece uma experiência de uso próxima à de sistemas HIS reais.

3. Entidades do Sistema

O sistema deve gerenciar, no mínimo, as seguintes entidades (a modelagem dos atributos é livre, mas os exemplos abaixo servem de referência):

Pacientes

Atributos sugeridos: *id, nome_completo, CPF, data_nascimento, sexo, telefone, email, endereço, convênio, número_carteirinha*

Profissionais

Atributos sugeridos: *id, nome, CRM/COREN/CRF, especialidade_id, cargo, turno, telefone, email, ativo*

Especialidades

Atributos sugeridos: *id, nome, descrição, área* (ex: *Clínica Geral, Cardiologia, Ortopedia*)

Atendimentos

Atributos sugeridos: *id, paciente_id, profissional_id, data_hora, tipo* (consulta/exame/internação), *status, diagnóstico, observações, valor*

Relacionamentos: *Profissional pertence a uma Especialidade; Atendimento referencia Paciente e Profissional.*

4. Requisitos — Backend (API RESTful)

4.1 Rotas mínimas por entidade

Método	Rota (exemplo)	Ação
GET	/api/pacientes	Listar todos
GET	/api/pacientes/:id	Buscar por ID
POST	/api/pacientes	Cadastrar
PUT	/api/pacientes/:id	Editar
DELETE	/api/pacientes/:id	Excluir
GET	/api/atendimentos?data=...	Filtrar por data

4.2 Requisitos técnicos

- Linguagem e framework: livre (Node.js/Express, Python/FastAPI, Java/Spring, PHP/Laravel, C#/.NET, etc.)
- SGBD: livre (PostgreSQL, MySQL, MongoDB, SQLite — usar ORM/ODM é recomendado).
- Retorno sempre em **JSON**; usar **HTTP status codes** corretos (200, 201, 400, 404, 500).
- Validação de dados na camada de serviço/controller.
- Tratamento de erros com mensagens descritivas.
- **Documentação da API:** Swagger/OpenAPI, Postman Collection ou README detalhado com exemplos de requisição.
- CORS habilitado para permitir consumo pelo frontend.

4.3 Diferenciais (bônus)

- Autenticação com JWT (login de usuário).
- Paginação e busca/filtro por parâmetros de query.
- Relacionamentos com eager/lazy loading (ex: atendimento retorna dados do paciente e profissional).
- Testes unitários ou de integração (Jest, Pytest, JUnit, etc.).
- Deploy em nuvem (Railway, Render, Fly.io, Vercel, Azure, AWS).

5. Requisitos — Frontend

5.1 Funcionalidades obrigatórias

- Tela de listagem para cada entidade (tabela/cards com paginação visual).

- Formulário de cadastro com validação no cliente (campos obrigatórios, formatos).
- Funcionalidade de edição: pré-preencher formulário com dados existentes.
- Exclusão com diálogo de confirmação.
- Feedback visual de ações: toasts/alertas de sucesso e erro.
- Indicador de carregamento (spinner/skeleton) durante chamadas à API.
- Layout responsivo — funcionar bem em desktop e mobile.

5.2 Tecnologias sugeridas

Categoria	Opções
Framework JS	React (recomendado), Vue.js, Angular, Svelte
UI / CSS	Bootstrap 5, Tailwind CSS, Material UI, Chakra UI
HTTP Client	Axios, Fetch API nativa, React Query
Roteamento	React Router, Vue Router
Estado global	Context API, Zustand, Redux Toolkit (opcional)
Build/bundler	Vite (recomendado), Create React App, Webpack

5.3 Telas mínimas esperadas

Tela	Descrição
Dashboard / Home	Cards com totais: nº de pacientes, atendimentos hoje, profissionais ativos
Pacientes	Listagem, busca por nome/CPF, cadastro, edição, exclusão
Profissionais	Listagem com especialidade, cadastro, edição, exclusão
Especialidades	CRUD completo — lista e formulário simples
Atendimentos	Listagem com filtro por data/status, detalhes, cadastro (vínculo paciente + profissional), edição

5.4 Diferenciais visuais (bônus)

- Dashboard com gráficos (Chart.js, Recharts) — ex: atendimentos por dia/semana.
- Modo escuro / tema claro-escuro alternável.
- Calendário de agendamentos integrado (FullCalendar ou react-big-calendar).
- Impressão / exportação de relatório de atendimentos em PDF.
- Autenticação no frontend (tela de login com JWT).

6. Estrutura Sugerida do Repositório

O repositório pode ser monorepo ou dois repos separados. Estrutura monorepo sugerida:

```
medcore-system/
├── backend/ # API (ex: Node/Express ou Python/FastAPI)
└── src/
```

```
■ ■■■ .env.example
■ ■■■ README.md # Como rodar + exemplos de endpoints
■■■ frontend/ # Aplicação React (ou outro framework)
■ ■■■ src/
■ ■■■ README.md
■■■ README.md # Visão geral do projeto
```

README obrigatório deve conter: descrição do projeto, tecnologias utilizadas, instruções de instalação e execução (passo a passo), variáveis de ambiente necessárias e exemplos de uso da API.

7. Critérios de Avaliação

Critério	Peso
API funcional com todas as rotas CRUD	25%
Frontend consome a API (integração real)	25%
Qualidade do código e organização	15%
Interface responsiva e usabilidade	15%
README e documentação da API	10%
Diferenciais / funcionalidades extras	10%

8. Forma de Entrega

- Entregar o **link do repositório público** (GitHub ou GitLab) com o código completo.
- O repositório deve estar **funcional**: clonar → seguir README → rodar sem erros.
- Caso haja deploy online (Vercel, Railway, etc.), incluir o link no README.
- Não serão aceitos repositórios privados ou com código somente em ZIP.